

Migrer par preuves et par cohortes

Le chemin critique est la frontière d'édition. Les migrations d'identité, de journal et de calcul avancent en parallèle, puis se rejoignent sur une installation complète sans JVM.

DÉCISIONS DÉJÀ FIXÉES

Les choix ouverts portent sur les mécanismes et les candidats, pas sur ces exigences

DÉCIDÉ

Zéro JVM · cores Rust · éditions autoritaires

WAP · publish-then-ack · journal durable · exécutants bornés

CHEMIN CRITIQUE

Un gate se ferme par une preuve exécutée et conservée

GATE OUVERT

1 · Frontière d'édition

Writer / publisher / preuves
Confidentialité et reprise

APRÈS GATE 1

2 · Premier EL réel

Source → raw → ack
C1–C4 corrigés, EL-1/2/3/6

PAR CONTRAT

3 · Cohortes métier

Icgres, Eidos et services
API, données, droits, reprise

ACCEPTATION FINALE

4 · Sortie JVM

Installation + exploitation
Restauration et workloads

QUALIFICATIONS EN PARALLÈLE

Pas de calendrier annoncé avant mesure des écarts et des workloads

CHOIX OUVERT

Identité

OIDC, tenants, droits, TTL

CHOIX OUVERT

Journal

Durabilité, replay, ack

CHOIX OUVERT

Calcul distribué

SQL, mémoire, DML, erreurs

Dépendances de migration, sans calendrier estimé · Tous les gates de qualification cible ci-dessus restent ouverts.

Migrer par preuves et par cohortes

Trajectoire proposée · 2 octobre 2026 · L'HTML associé contient tous les blocs, les scénarios et les références.

Le lot qui conditionne les autres

Priorité 1 · Construire la publication contrôlée et prouver ses refus.

Cas requis : première table privée, worker sans accès main, candidat rejeté illisible, CAS concurrent, schéma concurrent, fence zombie, perte de réponse et reprise à cache vide. Un simple verrou local ne ferme pas le gate.

Ce qui ne reste pas à arbitrer

Décision confirmée · La cible finale exclut toute JVM et conserve les six invariants.

EL-4 est tranché : extraction OLake Go encapsulée, writer/publisher Rust. Si le writer direct n'est pas disponible, journal privé puis raw sous WAP. Pas d'exception STREAM publiant avant audit.

Un vertical complet avant généralisation

Priorité 2 · Prouver une source réelle jusqu'à l'ack, avec fautes injectées.

EL-1/2/3/6 d'abord, EL-8 avant dérive automatique. Qualifier transactions complètes, TRUNCATE, bootstrap/CTID, checkpoint vide et source partagée. Le prototype Python fournit des contre-exemples à fermer, pas le service Rust final.

Une cohorte, un propriétaire d'écriture

Trajectoire proposée · Basculer des usages complets avec un point d'autorité explicite.

Conserver façades compatibles ; shadow read possible. Un seul propriétaire de schedule, publication et ack. Séparer rollback logiciel, données et effets externes. Les changements de schéma suivent Expand / Migrate / Contract.

Reprise

Rétablir clés, identités, commandes et journaux en plus des éditions. Reconstituer le checkpoint depuis les publications autorisées, puis vérifier la couverture des acks.

Gouvernance de décision

Chaque gate possède scénario, environnement épinglé, preuve conservée et décision. Un candidat reste ouvert tant que ces éléments manquent.

Sources détaillées dans l'HTML : Backlog de reprise · Trajectoire détaillée · Gates et campagnes.

Les mécanismes proposés ne sont pas des fonctionnalités déjà livrées. Dépendances de migration, sans calendrier estimé · Tous les gates de qualification cible ci-dessus restent ouverts.